

Apunte 06 — Cargar SPECTER y tu primer vector (usar vs. construir un modelo)

Proyecto Froth · aprender Machine Learning construyendo

1 Qué hicimos

En el Apunte 05 vimos la *idea* del embedding. Aquí lo tocamos de verdad: tomamos un paper real de tu tabla y lo convertimos en su vector.

1. Cargamos un paper (título + abstract) desde `2_Datos/processed/papers.parquet`.
2. Cargamos el modelo con una línea: `SentenceTransformer("allenai-specter")`.
3. Lo convertimos: `model.encode(texto)` → un vector de **768 números**.

El paper de prueba fue “*Electrostatically Controlled Enrichment of Lepidolite via Flotation*” y su vector empezaba por `[-0.59, -0.16, 1.34, ...]`. Ese paper es ahora un punto en el mapa del significado.

Concepto: modelo preentrenado, en la práctica

La primera vez que llamas a `SentenceTransformer("allenai-specter")`, la librería **descarga** el modelo (~440 MB) desde el repositorio Hugging Face y lo guarda en una *caché* en tu disco. Las siguientes veces lo carga de ahí, al instante, sin internet. No entrenas nada: usas pesos que otros ya calcularon.

Ojo / error típico

Título + abstract, no solo abstract. SPECTER fue entrenado para recibir el título pegado al abstract, porque el título aporta señal fuerte del tema. Por eso el código hace `texto = titulo + ". " + abstract` antes de vectorizar. Un detalle pequeño que mejora los resultados.

Ojo / error típico

Los *warnings* que salieron son inofensivos. Uno hablaba de “symlinks” (una optimización de disco en Windows) y otro de “HF_TOKEN” (una llave opcional para descargar más rápido). Ninguno impide nada: el modelo cargó y funcionó. Aprende a distinguir un *warning* (aviso, sigue adelante) de un *error* (se detiene).

2 La pregunta clave: ¿construimos nosotros el modelo?

No. Y conviene tener clarísima la diferencia entre dos cosas que suenan parecidas:

	USAR un modelo	ENTRENAR un modelo
¿Qué haces?	Descargas uno ya hecho y le pasas texto	Le enseñas desde cero con datos; ajustas sus “pesos”
¿Cuánto cuesta?	Una línea de código, segundos	Semanas, muchos datos, tarjetas gráficas (GPU)
¿En Froth?	Fases 2 a 5 (lo de ahora)	Fase 6 , opcional y avanzada

Concepto: por qué reutilizar es lo correcto (ahora)

Para dibujar las burbujas y el mapa, el SPECTER genérico —entrenado con millones de papers de todas las áreas— es más que suficiente. Entrenar tu propio modelo de embeddings desde cero sería reinventar la rueda, carísimo y sin ganancia clara en esta etapa. Regla del proyecto: *reciclar sin vergüenza*. Lo original de tu trabajo está en **cómo combinas** las piezas y el problema que resuelves, no en re-fabricar el motor.

3 Entonces, ¿cuándo sí tocamos “el modelo”?

En la **Fase 6** (la última, opcional). Ahí el experimento con potencial de tesis es *afinar* (fine-tuning) un modelo con papers de tu propio dominio (minerales, flotación, metalurgia) y medir si produce **mejores** clusters que el SPECTER genérico. Eso sí es “construir algo tuyo”, y para entonces habrás aprendido PyTorch y cómo funciona un modelo por dentro. Es el postre, no el plato principal.

En una frase

El modelo de vectores (SPECTER) ya está construido: solo lo descargamos y usamos, no lo entrenamos. Usar $\neq$ entrenar; entrenar lo tuyo es la Fase 6, opcional. Por ahora, tranquilidad: el motor ya está hecho.